

TASK 12

ACCOMPLISHED!



NEW SKILL UNLOCKED:
MONGODB



LEARNING HIGHLIGHTS:



INTRODUCTION TO MONGODB
(Non-Relational Documents)



JSON
(JavaScript Object Notation Data Format)



FOUNDATION OF QUERIES:
Fetching, Inserting, Sorting, Restriction, Deleting



COMPLEX QUERIES:
(\$in, \$nin, \$lt, \$lte, \$gt, \$gte, \$exists, \$set, \$unset)



SUBMITTED TO **TECH DATA COMMUNITY**



SUBMITTED BY: **PRIYADHARISHINI L**



DAY 23:

MongoDB:

C drive → mongosh
bin → mongosh

Introduction:

- ⇒ Document database.
- ⇒ It stores data in type of JSON [Javascript Object Notation] format
- ⇒ $\{ \text{key: 'value'} \}$
 ↓ ↓
 Question Answer
- ⇒ It is one of the unstructured DBMS.

Displaying the DB:

show dbs

admin
config
local

→ we should
not go inside
that

[We should use
lowercase here.
semicolon can be
put (or not put)]

To implement unstructured data we need 2 :-

⇒ MongoDB → app (like oracle10g)

⇒ mongoshell → like packages

↳ it is a command line interface
used to interact with MongoDB.

For creating a document: [like a table]

Use appdb

[switched to db appdb]

[A single
document has
in number of
collections but
it must have 1 collection]

[Here we are using users
collection; we can create
many collections]

After switching to appdb
we can create collections

⇒ Inserting one record in the collection:

```
db.users.insertOne({name: 'Priya'})
```

↓
collection name

↓
function

↓
key values

[For every insertion there it displays the object Id, along with index number & starts with 0]

```
db.users.insertOne({name: 'Ram',  
age: 23, salary: 5000})
```

(Here all the values are inserted; but we use insertOne; since it is unstructured it is inserted upon)

```
db.users.insertOne({area: 'madurai'})
```

Fetching the record:

```
db.users.find()
```

clearing the screen:

```
cls
```

⇒ Inserting many records in the collection:

```
db.users.insertMany([{}], [{}], [{}])
```

array key & values

function

[For inserting multiple values; we should use Array]

```
db.users.insertMany([ {name: 'Selva',  
  age: 24, salary: 24000}, {name: 'Raj',  
  rank: 1}, {name: 'Baba',  
  hobbies: ['eat', 'sleep', 'learn']} ])
```

④ defining the array inside another array

For displaying the records:

```
db.users.find()
```

[Data should be inserted using JSON format]

④ [Inside the JSON, we can insert another JSON]

Fetching the record with the specific condition:

```
db.users.find({name: 'Priya'})
```

```
db.users.find({age: 23})
```

```
db.users.find({rank: 1})
```

[In Oracle, we use where condition but in NRDBMS we just specify that key & value]

Fetching only the limited number of records:

```
db.users.find().limit(3)
```

→ Displaying the first 3 records that we inserted

```
db.users.find().limit(4)
```

Sorting out of records: [Ascending]

```
db.users.find().sort({name: 1})
```

Result: It displays the name of the records according to ascending order since we are mentioning it as 1;

Order $\Rightarrow$ 0 to 1

Whoever has no name displays first and followed by the name in Ascending order.

[Descending]

```
db.users.find().sort({name: -1})
```

Whoever has no name displays last first and followed by the name in descending order.

```
db.users.find().sort({age: -1})
```

```
db.users.find().sort({age: 1})
```

There are 2 names,
Ram, Raj sorting in
ascending order means
it will pick which name
should entry first.

Fetching results by applying both limit and sort:

```
db.users.find().sort({age: 1}).limit(2)
```

Applying the skip concept:

```
db.users.find().skip(2)
```

Applying skip & limit [to fetch the centre data scenario]

```
db.users.find().skip(4).limit(2)
```

DAY-24

Complex queries.

\$eq, \$ne, \$lt, \$gt, \$lte, \$gte, \$in, \$nin,
\$exists;

```
db.users.find({name: {$eq: 'Praveen'}})
```

```
db.users.find({name: {$ne: 'Praveen'}})
```

```
db.users.find({age: {$eq: 19}})
```

```
db.users.find({age: {$ne: 19}})
```

```
db.users.find({age: {$lt: 20}})
```

```
db.users.find({age: {$lte: 20}})
```

```
db.users.find({age: {$gt: 30}})
```

```
db.users.find({age: {$gte: 30}})
```

we fetch multiple so using of array

```
db.users.find({name: {$in: ['Praveen', 'Raj']}})
```

```
db.users.find({name: {$in: ['Praveen', 'Raj']}})
```

```
db.users.find({age: {$in: [19, 45]}})
```

```
db.users.find({age: {$in: [19, 45]}})
```

```
db.users.find({age: {$exists: true}})
```

If there is age key in the document then it will displays : so we are using exists there and if we want results use ☐ True or ☐ False

```
db.users.find({age: {$exists: false}})
```

```
db.users.findOne({age: {$gte: 20}})
```

```
db.users.countDocuments({age: {$gte: 20}})
```

```
db.users.countDocuments()
```

→ retrieve all the documents like in SQL count.

Updating the records:

```
db.users.updateOne({name: 'Ram'},  
{$set: {age: 50}})
```

```
db.users.updateOne({name: 'Selva'},  
{$set: {salary: 30000}})
```

```
db.users.updateOne({name:{$exists:true}},  
  {$set: {salary: 60000}})
```

↳ [This logic is wrong one; since we are mentioning updateOne it acts as a first come first serve basis; so it sees the first record if the record has name key then it update the value of 60,000. If we use updateMany here means it automatically changes the salary of 60000 wherever the name exists]

If we need to update the salary of 60,000 we use,
db.users.updateMany({salary:{\$exists:true}},
 {\$set: {salary: 60,000}})

Increment concept:

```
db.users.updateOne({name: 'Praveen'},  
  {$inc: {salary: 5000}})  
db.users.updateMany({salary:{$exists:true},  
  $inc: {salary: 5000}})
```

Rename: [we can rename the keys]

```
db.users.updateOne({user: 'unknown'},  
  {$rename: {user: "DATA"}})  
db.users.updateMany({name: {$exists: true}},  
  {$rename: {name: "FIRSTNAME"}})
```

(Wherever there is array, there we are having pull and push concept)

Array is different from JSON
[] { }

```
db.users.updateOne({ hobbies: { $exists: true } },  
  { $push: { hobbies: 'learn' } })
```

arrayname

Insert is different from Push & Pull.

```
db.users.updateOne({ hobbies: { $exists: true } },  
  { $pull: { hobbies: 'learn' } })
```

Removing the key and value → using unset:

```
db.users.updateOne({ $name: 'Raj' },  
  { $unset: { status: '' } })
```

Replace → completely erase the data

There is only replaceOne not replaceMany.

```
db.users.replaceOne({ $name: 'Selva' },  
  { user: 'unknown' })
```

```
db.users.updateMany({ $name: { $exists: true } },  
  { $unset: { $name: '' } })
```

ReplaceOne → data-നെ **completely** **erase** **the** **data**.

Deleting the records:

```
db.users.deleteOne({fname: 'Ram'})
```

```
db.users.deleteMany({age: {$exists: true}})
```

```
db.dropDatabase()
```

```
exit
```

[In delete it delete everything in the collection & but in unset it removes only that particular key & value]

deleteone

Name: Raj

Age : 23

Rank : 1

[delete name, age,
rank]

unset

Name: Raj

Age : 23

Rank : 1

[It removes the specific
key & value acc. to the
condition remaining only
rank: 1]